EcoFlux — Pipeline CI/CD e Containerização

Integrantes

- Alana de Menezes Pereira — RM 562562
- Carlos Eduardo Gonçalves de França — RM 561206
- Gustavo Cruz de Oliveira — RM 565633
- Nelson Trindade Teixeira — RM 563946
- Wendel de Santana Santos — RM 566354

1. Visão geral

O EcoFlux é uma API de gestão automatizada de resíduos. A aplicação registra pontos de coleta, descartes e coletas, mantendo alertas de capacidade para apoiar decisões operacionais ligadas à sustentabilidade.

A solução utiliza Java Spring Boot, autenticação JWT, migrações Flyway e H2 persistente em arquivo. O H2 foi escolhido para a demonstração DevOps porque o enunciado exige um serviço de banco no Compose, mas não exige Oracle. O banco fica em volume Docker para manter os dados entre reinícios.

2. Arquitetura



Na mesma VM, os ambientes usam diretórios, nomes de projeto Compose, containers, volumes e portas diferentes. Isso atende à separação operacional exigida para a demonstração; uma separação de VMs pode ser adotada se a infraestrutura estiver disponível.

3. Pipeline

O workflow `.github/workflows/ci-cd.yml` executa:

1. Checkout do código.
2. Configuração do Java 17.
3. Execução de `./mvnw -B test`.
4. Geração do JAR.
5. Construção e publicação da imagem no GitHub Container Registry em pushes para `main` ou tags de versão.
6. Deploy de staging após push na branch `main`, quando `ENABLE_STAGING=true`.
7. Deploy de produção após uma tag `vX.Y.Z`, quando `ENABLE_PRODUCTION=true`.

Os jobs de deploy começam desativados para impedir falhas antes da criação da VM e dos secrets.

4. Testes automatizados

O perfil `test` usa H2 em memória, carrega o contexto Spring e executa os testes das regras de negócio. Após a migração para H2 persistente, a validação local executou 5 testes, sem falhas ou erros.

```
./mvnw -B test

Resultado: BUILD SUCCESS — 5 testes executados, 0 falhas e 0 erros
```

5. Docker

O Dockerfile usa build em duas etapas. A primeira utiliza Maven com Java 17 para compilar o JAR. A segunda utiliza uma imagem JRE enxuta, executa com usuário sem privilégios e expõe a porta 8080.

O Compose executa a API, monta o volume `ecoflux-data` em `/data` e conecta o serviço à rede `ecoflux-network`. O H2 cria o arquivo do banco dentro do volume.

6. Execução local

```
cp .env.example .env
# editar DB_PASSWORD e JWT_SECRET
docker compose up --build -d
docker compose ps
curl http://localhost:8080/pontos-coleta
docker compose down
```

Para apagar o banco local junto com o Compose:

```
docker compose down -v
```

7. Deploy no Azure

A VM deve usar Ubuntu x64 e Docker Engine com Docker Compose. Para uma demonstração de baixo custo, staging e produção podem compartilhar a VM, usando:

- staging: `/opt/ecoflux-staging`, porta `8080`, projeto `ecoflux-staging`;
- produção: `/opt/ecoflux-production`, porta `8081`, projeto `ecoflux-production`.

Secrets esperados nos ambientes do GitHub:

```
HOST
USER
SSH_KEY
APP_DIR
APP_PORT
COMPOSE_PROJECT_NAME
DB_USER
DB_PASSWORD
JWT_SECRET
GHCR_READ_TOKEN
```

As variáveis de ativação são:

```
ENABLE_STAGING=true
ENABLE_PRODUCTION=true
```

Não devem ser usados prints simulados. Os campos abaixo devem ser preenchidos somente com evidências reais:

8. Evidências obrigatórias

8.1 Build e testes

- URL da execução do GitHub Actions: `PREENCHER`
- Resultado do job `Build e testes`: `PREENCHER`
- Resultado dos testes após a migração: `PREENCHER`

8.2 Imagem Docker

- URL ou nome da imagem no GHCR: `PREENCHER`
- Tag utilizada: `PREENCHER`

8.3 Staging

- URL ou IP do ambiente: `PREENCHER`
- URL da execução do deploy: `PREENCHER`
- Endpoint validado: `PREENCHER`
- Print do job e do ambiente: inserir evidência real

8.4 Produção

- URL ou IP do ambiente: `PREENCHER`
- Tag de release: `PREENCHER`
- URL da execução do deploy: `PREENCHER`
- Endpoint validado: `PREENCHER`
- Print do job e do ambiente: inserir evidência real

9. Decisões técnicas

Banco de dados

O enunciado não exige Oracle. A implementação utiliza H2 persistente para reduzir a infraestrutura necessária à demonstração e manter o Compose com banco, volume e rede. O H2 não deve ser tratado como escolha para alta escala, mas é suficiente para o escopo acadêmico e para demonstrar o ciclo DevOps.

Dois ambientes

Staging e produção possuem jobs separados, secrets separados e projetos Compose separados. Na demonstração econômica, os dois ambientes são executados na mesma VM em portas e diretórios diferentes.

Segredos

O repositório contém apenas `.env.example`. Senhas, JWT, chave SSH e token do GHCR

ficam nos secrets do GitHub.

10. Checklist

Item	Situação
Projeto compactado em `.zip`	Preparado
Dockerfile funcional	Preparado
`docker-compose.yml` com volume, variáveis e rede	Preparado
Pipeline de build	Preparado
Pipeline de testes	Preparado
Deploy automatizado em staging	Configurado; requer VM e secrets
Deploy automatizado em produção	Configurado; requer VM e secrets
README.md	Preparado
Documentação técnica	Este documento
Evidências reais de staging e produção	Pendente de execução